

How Akari remembers, decides when to speak, and answers

Akari is a Discord bot with a persistent personality. She keeps short-term and long-term memory, forms beliefs about people, tracks who is talking to whom, and only replies when she is actually being addressed. This guide covers how the two source files, `bot.js` and `embedder-worker.js`, fit together.

Every message	Save to history, decide whether to reply, build context, answer. Embeddings run in a worker thread.	cheap
Every ~10 messages	Extract durable facts, short-lived session state, and the occasional private thought.	cheap
Every ~150 messages, or after quiet time	Reflection: update beliefs, goals, user profiles, and rapport. Decay old memories.	moderate
Every ~1,200 messages	Major reflection: merge duplicate beliefs, reconsider who Akari is becoming, drop stale goals.	expensive

What the bot does, in plain terms

- Listens in one configured channel per server (set with `/setup`).
- Decides on its own whether a message is meant for her, even without an @mention, using a small "social brain" model call.
- Remembers things about people across days and weeks, and lets unimportant memories fade.
- Describes images people post, and can remember what they showed.
- Falls back gracefully at every layer: no Supabase, no embedder, or a failing model all degrade to simpler behavior instead of crashing.

Technology stack

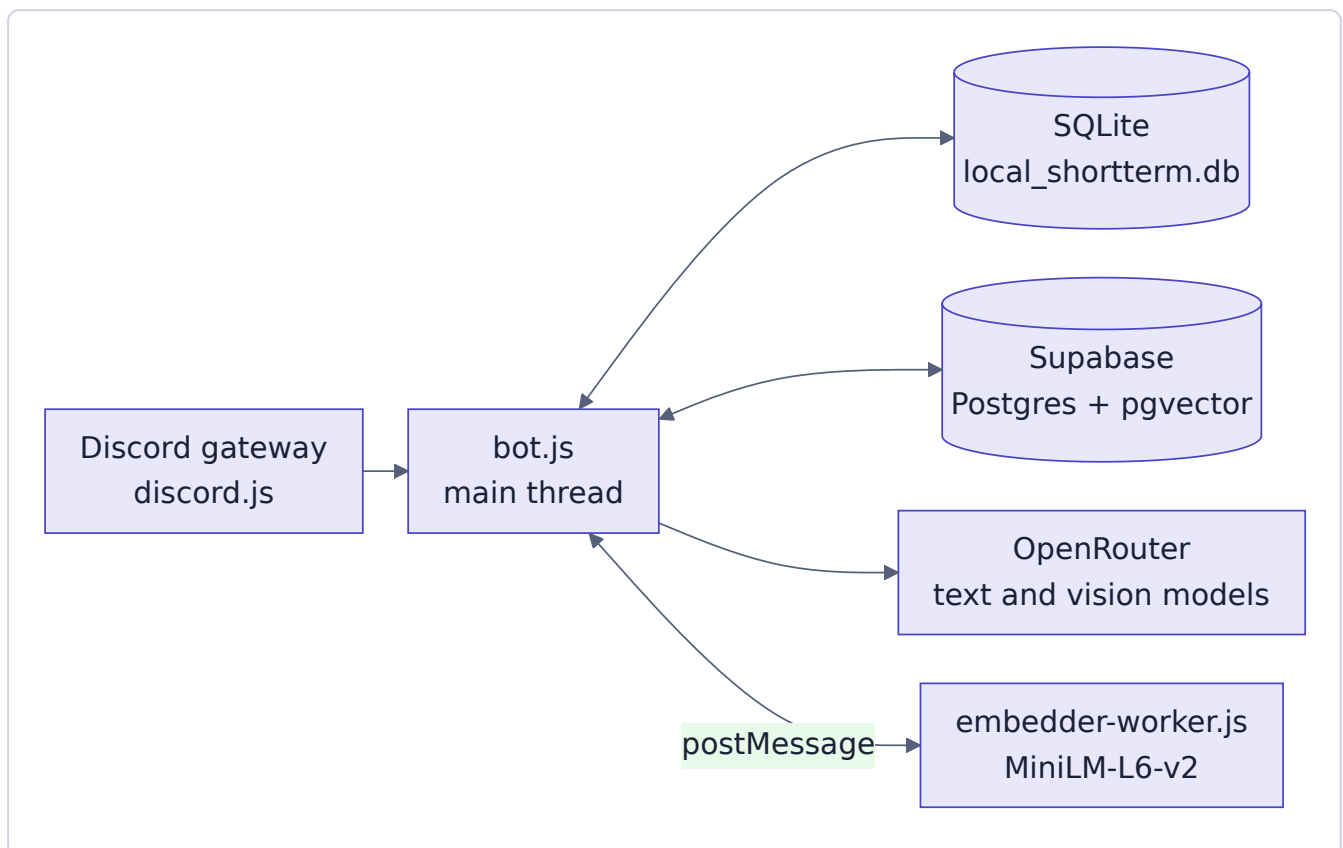
Every dependency is there for a specific job. The "benefit" column reflects the reasoning stated in the code comments or visible in how each piece is used.

Component	Role	Benefit
Node.js (ES modules)	Runtime for both files	Built-in <code>node:sqlite</code> and <code>node:worker_threads</code> mean no native database driver or threading library to install.
discord.js	Gateway connection, slash commands, message events	Handles heartbeats, reconnects, and rate limits. Keeping the main thread free protects the heartbeat.

Component	Role	Benefit
axios	HTTP calls to OpenRouter	One client for text and vision calls, with per-request timeouts (used on the vision path).
OpenRouter	Gateway to all language and vision models	One API key and one request format for many models, which makes primary/fallback switching a one-line change.
node:sqlite (DatabaseSync)	Local short-term storage (local_shortterm.db)	Zero-latency reads for the hot path: recent history, working memory, conversation threads, diagnostics.
Supabase (Postgres + pgvector)	Long-term memory, beliefs, goals, profiles, rapport	Durable cloud storage plus native vector similarity search, so semantic recall does not need a separate vector database.
Hugging Face Transformers.js	Runs Xenova/all-MiniLM-L6-v2 locally	Free, private embeddings with no API calls. Produces 384-dimension vectors.
Worker thread	Hosts the embedder	Matrix math (100 to 400 ms per call, per the code comments) never blocks the event loop or the gateway heartbeat.

System architecture

The bot is a single Node.js process with one helper thread. It talks to Discord, to OpenRouter, and to Supabase, and keeps its own local SQLite file.



Local brain

Fast, disposable, per-channel. SQLite holds raw history, session state, thread tracking, private thoughts, and diagnostics.

Cloud brain

Durable, per-server. Supabase holds what Akari has learned: memories, beliefs, goals, profiles, and rapport.

Language models

Reached through OpenRouter. Different models handle replies, quick structured tasks, and images.

Embedding worker

Turns text into vectors off the main thread so memory search stays cheap and the bot stays responsive.

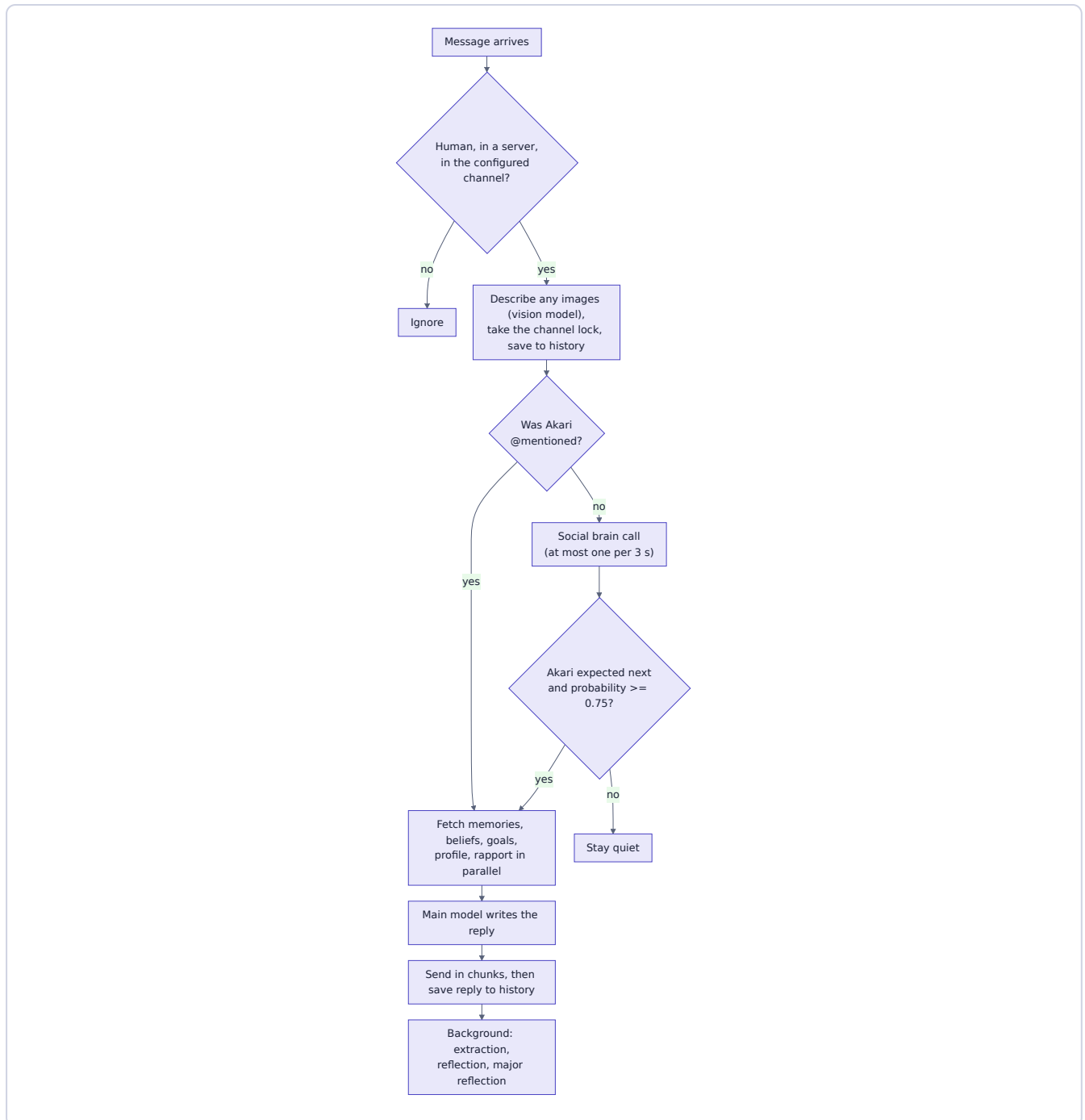
Data storage

Local SQLite tables

Table	Holds	Retention
configs	One active channel per server (<code>guild_id</code> , <code>channel_id</code>)	Until <code>/disable</code>
history	Every user message and every bot reply, with image URLs	Purged after 30 days, checked daily
working_memory	Short-lived session state, such as "currently studying for an exam"	Expires per row (default 30 min), deleted whenever it is read
conversation_threads	Social-brain output: participants, topic, expected next speaker, reply probability	Deleted after 60 idle minutes
reflection_log	Last reflection time per channel	Permanent
thought_stream	Akari's private one-line thoughts, feeding the next reflection	Consumed thoughts purged after 7 days
diagnostics_log	Telemetry events used by <code>/stats</code>	Purged after 30 days, checked daily

Message lifecycle

This is the path a single message takes, from arrival to (possibly) a reply.



- 1 Filter.** Bots, direct messages, and channels other than the configured one are ignored immediately.
- 2 Sanitize the name.** The display name is cleaned once (see [Security](#)) and that cleaned form is used everywhere downstream, including as a database key.
- 3 Look at images first.** Vision runs before the channel lock because it can take 5 to 12 seconds and touches no shared state. Holding the lock would stall every other message in the channel.
- 4 Lock and record.** A per-channel mutex serializes read-then-write work. Other channels are unaffected.

- 5 **Decide whether to speak.** An @mention is an automatic yes. Otherwise the social brain decides (next section).
- 6 **Build context only when replying.** Retrieval is skipped for the many messages Akari stays quiet on.
- 7 **Reply, then record.** The message is sent to Discord before it is saved to history, so local history never claims she said something the user did not receive.
- 8 **Housekeeping.** Extraction and reflection are fire-and-forget and never delay the reply.

Social brain

Instead of answering every message or only answering @mentions, Akari runs a small, fast model over the last 20 messages and asks one question per conversation thread: who is the natural next speaker?

A channel can hold several conversations at once. Each becomes a thread with participants, a topic, a status (`active` , `paused` , `idle`), an `expected_next_speaker` , a `reply_probability` , a confidence, and a one-sentence reason. Threads are keyed by the sorted, lowercased participant list and stored with an upsert.

Situation	What happens
Message @mentions Akari	Reply is decided immediately. Threads are read from disk for context, and refreshed in the background for next time.
Another social-brain call happened under 3 s ago	The call is skipped and the last stored thread state is used. No message is lost, since the transcript is always built from full history.
Otherwise	The social model runs, threads are updated, and the speaker's thread is looked up.
Gatekeeper	Reply only if the speaker's thread expects Akari next and <code>reply_probability</code> is at least 0.75.

The rate limit is deliberately not a keyword filter. A keyword heuristic would miss the implicit cases the social brain exists to catch, such as someone replying to Akari's last line without naming her. Threads with no activity for 60 minutes are deleted.

Language models

Role	Model	Used for
Primary	<code>deepseek/deepseek-v4-flash-0731</code>	The main reply, reflection, and major reflection. These benefit from real reasoning.
Fallback	<code>nvidia/nemotron-3-ultra-550b-a55b:free</code>	Automatically used when any other text model call fails.
Social	<code>openai/gpt-oss-20b:free</code>	Cheap structured work: social brain, memory extraction, vision-to-memory, memory merging. Runs with reasoning effort <code>low</code> so the reasoning trace cannot eat the token budget and truncate the JSON.
Vision (primary)	<code>nvidia/nemotron-3-nano-omni-30b-a3b-reasoning:free</code>	Turns image attachments into a short text description.

Role	Model	Used for
Vision (fallback)	google/gemma-4-31b-it:free	Second attempt if the primary vision call fails or times out.

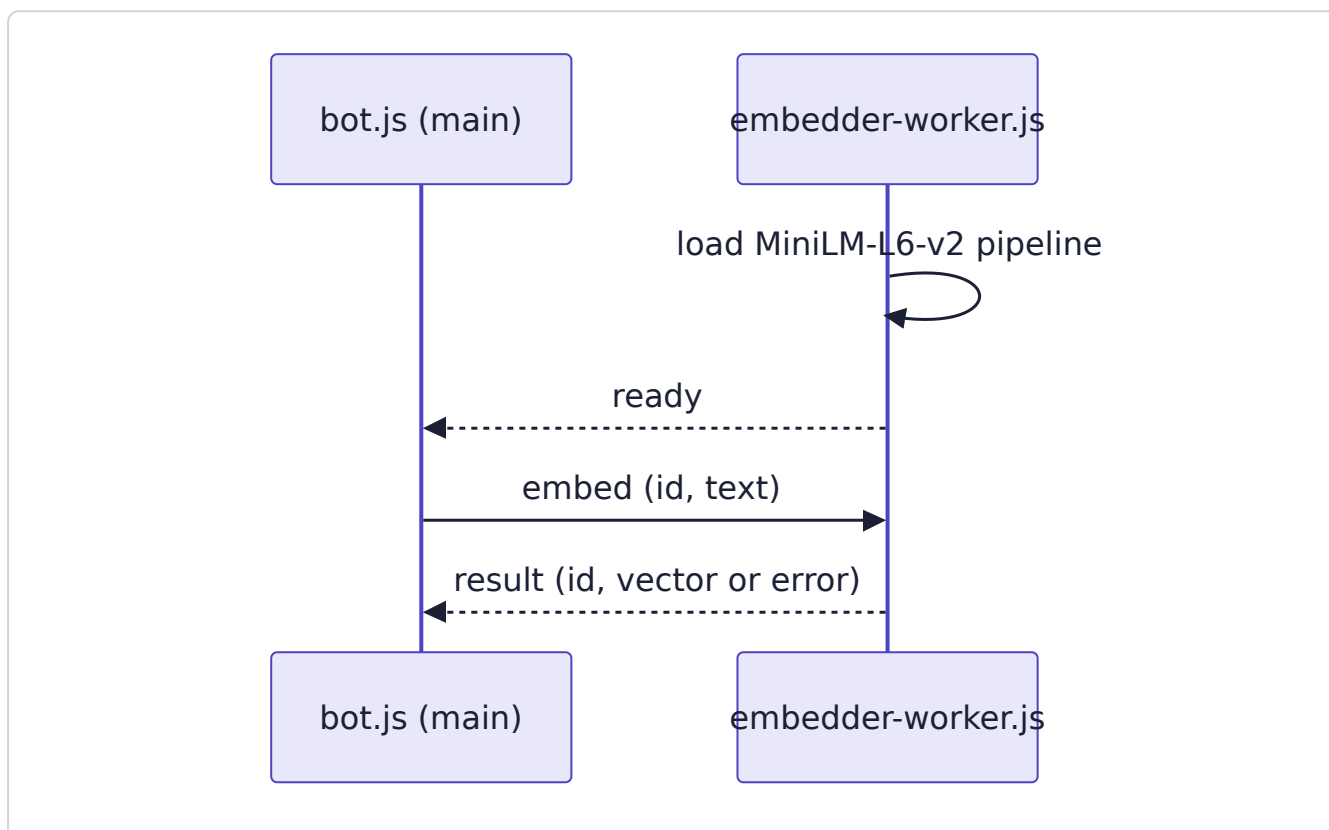
How callLLM works. Every text call posts to OpenRouter with temperature 0.7 and a 1,500-token cap. JSON mode and reasoning effort are added only when requested. If the call throws and the model was not already the fallback, it logs an `llm_fallback` diagnostic and retries once with the fallback model. If the fallback also fails, the error propagates to the caller.

Robust JSON. `safeParseJSON` strips code fences, extracts the outermost `{ ... }`, tries to parse, and if that fails replaces control characters and tries once more. A total failure returns `null`, and callers log a parse failure and move on.

The code notes that OpenRouter's free-model catalog rotates constantly. Verify each slug at openrouter.ai/models before deploying.

Embedding worker

Semantic search needs text turned into vectors. Doing that on the main thread would freeze the event loop for hundreds of milliseconds per call, which risks delaying Discord's heartbeat. `embedder-worker.js` moves the work to a dedicated thread.



Message	Direction	Payload
<code>ready</code>	worker to main	Model loaded. Main sets <code>embedderReady = true</code> .
<code>init_error</code>	worker to main	Model failed to load, with an error message. Main logs a warning.
<code>embed</code>	main to worker	<code>{ id, text }</code>
<code>result</code>	worker to main	<code>{ id, vector }</code> or <code>{ id, error }</code>

Design details

- **Request matching.** Each request gets an incrementing id. A `Map` of pending promises lets many embeds be in flight at once and resolves each to its own caller.
- **Vector format.** Mean pooling with normalization yields a 384-number array, sent back as a plain JavaScript array so it can go straight into pgvector.
- **Fail-safe contract.** `embedText()` returns `null` if the worker is not ready, has crashed, errors, or takes more than 10 seconds. Every caller treats `null` as "no embedding available": retrieval falls back to recent memories, consolidation is skipped, and belief matching creates new beliefs.
- **Lazy import.** The Transformers.js library is imported inside the worker, so a failed install or model download becomes an `init_error` message rather than a crash of the main process. The model files are typically downloaded on first run and cached, so the first start needs network access.
- **Start-up ordering.** The worker is launched from the `clientReady` handler, so embeddings are unavailable for a short window after login.

Memory system

Akari has four layers of memory, each with its own lifespan and storage.

Layer	Lives in	Lifespan	How it is written
Short-term history	<code>SQLite</code>	30 days on disk. The prompt sees only what fits a 9,000-token budget (hard cap 80 messages).	Every message and reply is saved.
Working memory	<code>SQLite</code>	30 minutes by default. The extractor can set a different TTL per item.	Extraction, for temporary situations.
Long-term memory	<code>Supabase</code>	Until it decays to archived or forgotten.	Extraction and vision, consolidated with near-duplicates.
Beliefs and goals	<code>Supabase</code>	Until pruned by decay, cap, or staleness.	Reflection cycles.

Short-term history

A flat "last N messages" limit treats a one-word reply and a long paragraph the same. Instead, `getShortTermHistory` reads up to 80 recent messages newest-first and keeps adding older ones until an estimated 9,000-token budget is reached (characters divided by four, plus 8 per message for framing). Results are returned oldest-first.

Extraction every ~10 messages

A cheap model reads the current history window and returns strict JSON with three parts: `long_term` entries, `working` entries, and a rarely-present `private_thought`.

Each long-term entry has a subject, a third-person summary, a `nature` (`fact`, `preference`, `relationship`, or `event`), an importance from 0 to 1, and a confidence from 0 to 1. Events are phrased narratively ("Last week, Zandar showed Akari photos of his cat") so they can be recalled as shared moments. Private thoughts are stored locally and feed the next reflection as a light hint.

Because the window overlaps between runs, the same fact will be seen repeatedly. That is why consolidation exists.

Consolidation

Before inserting a memory, the bot asks Postgres for the nearest existing memory with the same server, `nature`, and subject, at cosine similarity 0.86 or higher. If one exists, a cheap model merges the two statements into one sentence, the embedding is recomputed, importance rises by 0.08 (capped at 1), the evidence count increments, and status returns to `active`. So "likes astronomy", "enjoys looking at stars", and "interested in space" become one stronger memory.

Events are never merged. Two similar events are usually two separate occurrences, each worth keeping as its own callback.

Retrieval and ranking

When Akari is about to reply, the query `"Name: message"` is embedded and pgvector returns the 15 nearest memories. These are filtered and re-ranked in JavaScript:

```
keep only: similarity >= 0.35 and confidence >= 0.30

score = 0.50 * similarity
       + 0.25 * importance
       + 0.15 * exp(-daysSinceAccess / 20) // recency
       + 0.10 * ln(1 + accessCount) / ln(11) // usefulness, capped at 1

score += 0.08 if nature is "event" // stories surface more readily
score += 0.06 if memory's subject is the speaker // who is talking matters

return the top 8
```

A favorite hobby therefore outranks "ate pizza once" even if pizza is a slightly closer text match. The result count varies naturally: sometimes zero, sometimes several, with no extra model call needed to decide.

Returned memories are reinforced through `reinforce_memories` in the background, so useful memories stay alive. If the embedder or vector search is unavailable, retrieval falls back to the most recent memories.

Beliefs, goals, and rapport

Memory records what happened. Reflection is where "every conversation changes you a little" actually happens. It runs in three tiers so the expensive work stays rare.

Tier	Trigger	What it does
Extraction	History count reaches a multiple of 10	Raw facts, working memory, private thoughts.
Reflection	Multiple of 150 messages, or a 30-minute timer finds a channel that has had new messages but has not been reflected on in 180 minutes	Decay memories, update beliefs, goals, profiles, and rapport.
Major reflection	Multiple of 1,200 messages	Merge duplicate beliefs, reflect on self-beliefs using the top 25 memories server-wide, drop goals untouched for 21 days.

Beliefs

A belief is a durable impression with a scope: **user** (about one person), **server** (about the community), or **self** (about Akari). The reflection model proposes updates, each marked **reinforce**, **weaken**, or **new**, and can cite memory ids as evidence.

The bot does not trust the "new" label. It embeds the statement and looks for an existing belief in the same scope and subject with cosine similarity of at least 0.83. A match is treated as the same belief. Confidence then moves a fraction of the way toward 1 or 0 rather than jumping:

```
reinforce: c = c + (1 - c) * 0.18
weaken:    c = c * (1 - 0.15)
new belief: c = 0.45
```

Example, reinforced three times from 0.45:
0.45 -> 0.549 -> 0.630 -> 0.697

Passive decay. Beliefs nobody touched in a cycle lose 5% of their confidence. If fewer than half of their evidence memories are still alive, they lose 15% instead. Anything below 0.15 is deleted, and only the 25 strongest beliefs per server are kept.

Merging duplicates. Phrasing drift can let two near-identical beliefs coexist. Major reflection scans pairs (a quadratic scan, trivial at 25 beliefs), keeps the stronger one, gives it a small boost, moves the evidence links across, and deletes the other.

Goals

Goals are small things Akari is curious about or hoping to do, at most five per server. Each cycle the model returns a complete revised list. The new rows are inserted first, and the old rows are deleted by id only after the insert succeeds, so a failure never leaves a server with zero goals. Goals appear in the prompt as "Pending Goals" so she can raise them naturally.

Profiles and rapport

For each user she learned something new about, reflection writes a one-paragraph profile and a rapport value between 0 and 1, plus a short natural-language "current read" such as "curious and a bit playful lately".

Rapport is a single scalar that decays toward a 0.5 baseline with a 10-day half-life, so a relationship that goes quiet slowly cools back to neutral. When building a prompt the stored value is decayed to the present moment, then mapped to a sentence:

Rapport	Text given to the model
0.8 and above	Very close and comfortable, trust built over many conversations.
0.6 to 0.8	Warm and at ease.
0.4 to 0.6	Still getting to know them.
0.2 to 0.4	Still a little new.
Below 0.2	Doesn't know them well yet.

Prompt assembly

Only after the gatekeeper approves a reply does the bot build the system prompt. It is the fixed persona prompt plus these sections, appended in order:

1. **Working memory:** current session state.
2. **Subconscious social context:** thread participants, topic, and the social brain's reason.
3. **Long-term memories:** events as "You recall: ...", others tagged by nature.
4. **Beliefs:** the current speaker's user-scoped beliefs plus all server and self beliefs, framed as a worldview that colors behavior rather than facts to recite.
5. **Pending goals.**
6. **User profile** for the current speaker.
7. **Sense of this relationship:** decayed rapport text and the current read.

Memories, beliefs, goals, profile, and rapport are fetched together with `Promise.all` since none depends on another. Conversation history follows as chat turns. User turns are prefixed with `Name:` so she can tell speakers apart, but her own past replies are sent unprefixd. Prefixing them would teach the model a pattern it never produced, and models reliably start copying it.

The persona prompt itself defines a gentle, curious character who lives in a computer laboratory, values kindness and honesty over cleverness, avoids Unicode emoji, and uses kaomoji and small italic actions sparingly. It ends with a "golden rule": ask what Akari would genuinely think or wonder in this moment, then answer naturally.

Vision pipeline

Image attachments are converted to text by a dedicated vision model, so the main text model never needs to be multimodal. The vision model gets a system prompt asking for a 2 to 4 sentence summary under 120 words, covering the scene, any readable text, and mood, with temperature 0.3, a 250-token cap, and a 12-second timeout.

If the primary vision model fails, a `vision_fallback` event is logged and the fallback model is tried. If both fail, Akari receives the placeholder line "(Akari attempted to look at the image, but couldn't load it clearly.)" and the conversation continues.

The description is added to the stored message as a "Visual Context" block. A second, separate call then asks whether the image reveals a durable fact about the person (a pet, an object they own, a place) and stores it immediately, since a one-off image will often not come up again.

Reliability and concurrency

Technique	Where	Why
Per-channel mutex	<code>withChannelLock</code>	Prevents races between reading and writing history and thread state. Different channels run in parallel.
Fire-and-forget background work	Extraction, reflection, reinforcement, vision memory	None of it delays the reply.
Send first, record second	Reply path	Local history never contains a reply the user did not receive.
Typing keep-alive	<code>startTypingKeepAlive</code>	Discord's typing indicator lasts about 10 s. It is renewed every 8 s and always stopped in a <code>finally</code> .
Message chunking	<code>chunkForDiscord</code>	Splits at 2,000 characters, preferring paragraph, then line, then word boundaries. First chunk is a reply, the rest are plain sends.
Friendly error reply	Main handler	If context building or the model fails, the user gets an in-character message instead of silence.
Global handlers	<code>unhandledRejection</code> , <code>uncaughtException</code>	Log and keep the process alive.

Graceful degradation

If this is missing or fails	The bot does this instead
Supabase not configured	Warns at start. Long-term memory, beliefs, goals, and reflection are disabled. Chat and the social brain still work.
Embedder not ready, crashed, or slow	<code>embedText</code> returns <code>null</code> . Retrieval uses recency, consolidation and belief matching are skipped.
Vector search RPC error	Falls back to the most recent memories.
Text model failure	Retries once on the fallback model.
Vision failure on both models	Placeholder description, chat continues.
Model returns bad JSON	Logged as a parse failure and skipped for that cycle.

Security and privacy

Name sanitization

Discord nicknames are attacker-controlled and are pasted straight into prompts, including flattened transcripts with no role boundary. `sanitizeDisplayName` is the single place a name is read. It removes newlines, tabs, colons, and angle brackets, trims to 80 characters, and appends "(user)" to the reserved names `akari`, `system`, and `assistant`. The cleaned form is also the key for beliefs, profiles, and rapport.

Access control

The three slash commands require the Manage Server permission, and their replies are ephemeral. The bot only reads one channel per server and ignores other bots.

What is shared across users

User-scoped beliefs, profiles, and rapport are only shown when that person is speaking. Long-term memories are different: retrieval is per server, not per person, so a memory about one user can surface while talking to another if it is semantically relevant. Whether that is desirable depends on your community.

Configuration reference

All tunables are constants at the top of `bot.js`. Several were educated guesses, and `/stats` exists to help tune them against real usage.

History, extraction, and reflection

Constant	Value	Meaning
<code>SHORT_TERM_TOKEN_BUDGET</code>	9000	Estimated tokens of history sent to the model.
<code>SHORT_TERM_MAX_MESSAGES</code>	80	Hard cap on history messages.
<code>EXTRACTION_INTERVAL</code>	10	Messages between memory extractions.
<code>WORKING_MEMORY_TTL_MINUTES</code>	30	Default life of a working-memory item.
<code>REFLECTION_MESSAGE_INTERVAL</code>	150	Messages between reflections.
<code>REFLECTION_MIN_INTERVAL_MINUTES</code>	180	Minimum gap for the quiet-channel timer trigger.
<code>REFLECTION_TIMER_CHECK_MS</code>	30 min	How often the background timer checks.
<code>MAJOR_REFLECTION_MESSAGE_INTERVAL</code>	1200	Messages between major reflections.
<code>MAJOR_REFLECTION_STALE_GOAL_DAYS</code>	21	Untouched goals older than this are dropped.
<code>MAX_ACTIVE_BELIEFS</code>	25	Belief cap per server.
<code>MAX_ACTIVE_GOALS</code>	5	Goal cap per server.

Retrieval and beliefs

Constant	Value	Meaning
<code>MEMORY_CANDIDATE_COUNT</code>	15	Candidates fetched from pgvector before re-ranking.
<code>MEMORY_SIMILARITY_FLOOR</code>	0.35	Minimum similarity to be considered.
<code>MEMORY_CONFIDENCE_FLOOR</code>	0.3	Minimum confidence to be considered.
<code>MEMORY_MAX_RETURN</code>	8	Memories placed in the prompt.
<code>CONSOLIDATION_SIMILARITY_THRESHOLD</code>	0.86	Similarity that triggers a merge.
<code>EVENT_RETRIEVAL_BOOST</code>	0.08	Score bonus for events.
<code>SPEAKER_SUBJECT_BOOST</code>	0.06	Score bonus for memories about the speaker.
<code>BELIEF_MATCH_THRESHOLD</code>	0.83	Similarity to treat two statements as one belief.
<code>BELIEF_LEARNING_RATE</code>	0.18	How far reinforcement moves confidence toward 1.
<code>BELIEF_DECAY_RATE</code>	0.15	How far weakening moves confidence toward 0.
<code>BELIEF_NEW_STARTING_CONFIDENCE</code>	0.45	Initial confidence of a new belief.
<code>BELIEF_PASSIVE_DECAY</code>	0.05	Loss per idle cycle.
<code>BELIEF_EVIDENCE_FADE_DECAY</code>	0.15	Loss per idle cycle when most evidence has faded.
<code>BELIEF_PRUNE_THRESHOLD</code>	0.15	Beliefs below this are deleted.
<code>RAPPORT_BASELINE</code>	0.5	Value rapport decays toward.
<code>RAPPORT_DECAY_HALFLIFE_DAYS</code>	10	Half-life of rapport decay.

Social brain, retention, and limits

Constant	Value	Meaning
<code>SOCIAL_BRAIN_MIN_INTERVAL_MS</code>	3000	Minimum gap between social-brain calls per channel.
<code>THREAD_IDLE_MINUTES</code>	60	Idle threads are deleted after this long.
<code>REPLY_PROBABILITY_THRESHOLD</code>	0.75	Minimum probability for an unprompted reply.
<code>HISTORY_RETENTION_DAYS</code>	30	Raw history purge age.
<code>DIAGNOSTICS_RETENTION_DAYS</code>	30	Telemetry purge age.
<code>THOUGHT_STREAM_CLEANUP_DAYS</code>	7	Consumed thoughts purge age.
<code>HISTORY_CLEANUP_INTERVAL_MS</code>	24 h	Frequency of history purge.
<code>DISCORD_MAX_MESSAGE_LENGTH</code>	2000	Discord's per-message limit.

Commands and diagnostics

Command	Permission	Effect
<code>/setup channel:</code>	Manage Server	Sets the one channel where Akari is active in this server.
<code>/disable</code>	Manage Server	Removes the server's configuration so Akari goes silent.
<code>/stats</code>	Manage Server	Shows a diagnostics report, split into chunks if long.

Diagnostics events are written to `diagnostics_log` as JSON: `social_decision`, `main_reply`, `memory_retrieval`, `memory_consolidation`, `extraction_result`, `reflection_result`, `major_reflection_result`, `llm_fallback`, and `vision_fallback`. `/stats` reads them back and reports:

- Reply rate, split into mentions versus thread-probability replies, and average thread probability.
- Average main-reply latency.
- Average memories returned per reply, the zero-result rate, and the consolidation merge rate.
- Extraction and reflection run counts with JSON parse-failure rates.
- Global model and vision fallback counts.
- Live Supabase counts of active and archived memories, beliefs, and goals.

Setup and deployment

Requirements

- A Node.js version that includes the built-in `node:sqlite` module (22.5 or newer).
- A `package.json` with `"type": "module"`, since both files use `import` syntax.
- Packages: `discord.js` (a version that emits `clientReady`), `axios`, `@supabase/supabase-js`, `@huggingface/transformers`.
- An OpenRouter API key, a Discord bot token, and (optionally) a Supabase project with the pgvector extension.
- `embedder-worker.js` in the same folder as `bot.js`.

- 1 **Create the Discord application.** Add a bot user and enable the **Message Content** privileged intent. The bot also uses the Guilds and Guild Messages intents.
- 2 **Invite it** with permission to view the channel, send messages, and use slash commands.
- 3 **Fill in the four secrets** near the top of `bot.js`: Discord token, OpenRouter key, Supabase URL, and Supabase key. Move these to environment variables before sharing the code.
- 4 **Prepare Supabase** (optional but needed for long-term memory): enable pgvector and apply your schema SQL (the tables and the vector-search functions the bot calls).
- 5 **Verify model slugs** on OpenRouter, since free models come and go.
- 6 **Run** `node bot.js`. Watch for the "Cognitive Engine online" and "MiniLM-L6-v2 initialized" log lines.
- 7 **Run** `/setup` in your server and pick the channel.